PBCST304

OBJECT ORIENTED PROGRAMMING

MODULE - 3

Prepared by: Aswathy.T.K

Packages and Interfaces –

Packages - Defining a Package, CLASSPATH, Access Protection, Importing Packages.

Interfaces - Interfaces v/s Abstract classes, defining an interface, implementing interfaces, accessing implementations through interface references, extending interface(s).

Exception Handling - Checked Exceptions, Unchecked Exceptions, ***try*** Block and ***catch*** Clause, Multiple catch Clauses, Nested ***try*** Statements, ***throw***, ***throws*** and ***finally***, Java Built-in Exceptions, Custom Exceptions.

Introduction to design patterns in Java : Singleton and Adaptor.

Types of Errors in Java

Error Type	Description
Compile-Time Error	Detected by the compiler. Prevents code from running.
Runtime Error	Occurs while the program is running. Often causes crashes.
Logical Error	Code runs but gives incorrect results. Hardest to find.

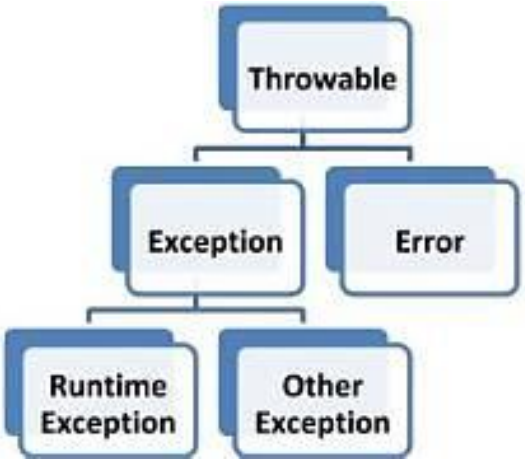
- When an error occurs, Java will normally stop and generate an error message.
- The technical term for this is: **Java will throw an exception** (throw an error).

EXCEPTION HANDLING:

- An exception is an **abnormal condition/event that arises in a code sequence at run time and affects the normal control of the flow of execution.**
- In other words, an exception is a **run-time error.**
- In computer languages that do not support exception handling, errors must be checked and handled manually. => This is troublesome.
- **Java provides exception handling mechanisms to effectively handle exceptions.**

Exception Classes:

- All exception classes are child classes of the **Throwable** built-in class.
- Therefore, exception classes are inherited from the Throwable built-in class.
- This throwable has two main branches: **Exception** and **Error.**
- **Runtime Exception** is the child class inherited from the parent class **Exception**.
- All Exception classes are defined in the **java. lang package**.



TYPES OF EXCEPTIONS

- In Java, exceptions are broadly classified into three types:

    1. Checked Exceptions (Compile-time Exceptions)

    2. Unchecked Exceptions (Run-time exceptions)

    3. Errors

Checked Exceptions (Compile-time Exceptions)

- These exceptions are **checked by the compiler.**
- The **programmer must** either **handle them** using try-catch or declare them using the throws keyword.
- They are subclasses of Exception (excluding RuntimeException).
- Examples:
  - IOException
  - SQLException
  - FileNotFoundException
  - ClassNotFoundException

Unchecked Exceptions (Compile-time Exceptions)

- These **exceptions occur during program execution.**
- The **compiler does not check them.**
- They are subclasses of RuntimeException.
- Examples:
  - ArithmeticException
  - NullPointerException
  - ArrayIndexOutOfBoundsException
  - NumberFormatException

Errors:

- Errors represent serious problems that applications generally should not try to handle.
- They are subclasses of **Error.**
- They are **caused by the Java Virtual Machine (JVM) or system-level failures.**
- Examples:
  - OutOfMemoryError
  - StackOverflowError
  - VirtualMachineError

EXCEPTION HANDLING KEYWORDS:

- There are five distinct keywords used for exception handling;
  - **try**
  - **catch**
  - **finally**
  - **throw**
  - **throws**

try:

- The **try** block contains statements that may generate an exception.

catch:

- The **catch** block handles the exception generated in the try block.
- Immediately following the try block, include a catch block that specifies the exception type that you wish to catch.

Syntax:

```
try
{
}
catch(Exception exceptionobject)
{
}
```

Eg: ArithmeticException generated by the division-by-zero error

```
class Exc2 {
  public static void main(String args[]) {
    int d, a;

    try { // monitor a block of code.
      d = 0;
      a = 42 / d;
      System.out.println("This will not be printed.");
    } catch (ArithmeticException e) { // catch divide-by-zero error
      System.out.println("Division by zero.");
    }
    System.out.println("After catch statement.");
  }
}
```

OUTPUT:

Eg: ArithmeticException generated by the division-by-zero error

```
class Exc2 {
  public static void main(String args[]) {
    int d, a;

    try { // monitor a block of code.
      d = 0;
      a = 42 / d;
      System.out.println("This will not be printed.");
    } catch (ArithmeticException e) { // catch divide-by-zero error
      System.out.println("Division by zero.");
    }
    System.out.println("After catch statement.");
  }
}
```

OUTPUT:

```
Division by zero.
After catch statement.
```

Note:

- In a *try* - *catch* construct, **once the *catch* statement has executed, program control continues with the next line in the program following the entire try/catch mechanism.**
- **A *try* and its *catch* statement form a unit.**
- **The scope of the *catch* clause is restricted to those statements specified by the immediately preceding *try* statement.**
- A *catch* statement cannot catch an exception thrown by another *try* statement.
- The statements that are protected by *try* must be surrounded by curly braces.
    - ie, they must be within a block.
- The goal of most well-constructed *catch* clauses should be to resolve the exceptional condition and then continue on as if the error had never happened.

Multiple *catch* Clauses:

- **In some cases, more than one exception could be raised** by a single piece of code.
- To handle this type of situation**, we can specify two or more *catch* clauses**, each catching a different type of exception.
- **When an exception is thrown, each *catch* statement is inspected in order, and the first one whose type matches that of the exception is executed.**
- After one *catch* statement executes, the others are bypassed, and execution continues after the *try/catch* block.

Eg:

```
class MultipleCatchDemo {

        public static void main(String[] args)
        {
        try
        {
        int a = 10;
        int b = 0;
        int result = a / b;   // ArithmeticException
        int arr[] = {10, 20, 30};
        System.out.println(arr[5]);  //ArrayIndexOutOfBoundsException
        }
     catch (ArithmeticException e)
     {
        System.out.println("Cannot divide by zero");
     }
     catch (ArrayIndexOutOfBoundsException e)
     {
        System.out.println("Array index is out of range");
     }
     catch (Exception e)
     {
        System.out.println("Some other exception occurred");
     }
   }
}
```

OUTPUT:

Cannot divide by zero

- The try block contains statements that generate exceptions.
- The first catch handles ArithmeticException, such as division by zero.
- The second catch handles ArrayIndexOutOfBoundsException, such as accessing an invalid array index.
- The third catch handles other exceptions that are not handled by the previous catch blocks.

- **<u>Note</u>: Only one catch block is executed for a particular exception.**
- Consider the example, if:

```
int result = 10 / 0;
```

- The above statement causes an ArithmeticException.
- So, Java directly executes:

```
catch (ArithmeticException e)
```

- The remaining catch blocks won't get execute.

<u>Rule for Multiple Catch:</u>

- **The more specific exceptions must come before more general exception.**

<u>Eg:</u>

```
catch (ArithmeticException e)

catch (Exception e)
```

- The statement:

  int result = a / b; has b = 0, so it causes an ArithmeticException.

- Java immediately transfers control to:

```
catch (ArithmeticException e)
{
   System.out.println("Cannot divide by zero");
}
```

- After executing this catch block, the program skips the remaining catch blocks.
- So the statement:

  System.out.println(arr[5]); // This is never executed.

Final Output:

Cannot divide by zero

Nested try Statements

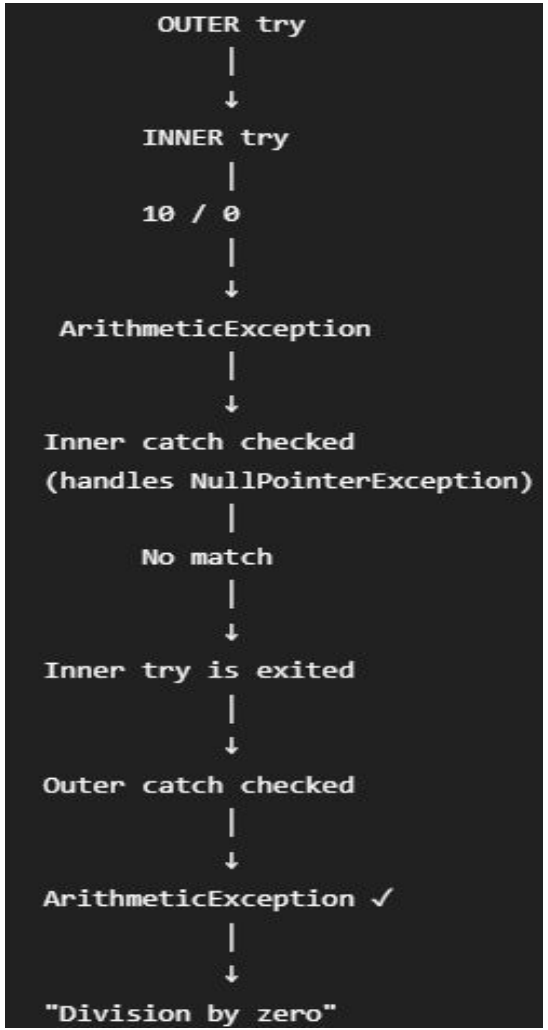
- The *try* statement can be nested.
- ie, a *try* statement can be inside the block of another *try*.
- Each time a *try* statement is entered, the context of that exception is pushed on the stack.
- If an inner *try* statement does not have a *catch* handler for a particular exception, the stack is unwound and the next *try* statement's catch handlers are inspected for a match.
- This continues until one of the *catch* statements succeeds, or until all of the nested *try* statements are exhausted.
- If no *catch* statement matches, then the Java run-time system will handle the exception.

Eg:

```
class NestedTryDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try");
            try {
                System.out.println("Inner try");
                int a = 10 / 0;   // ArithmeticException
                 }
                 catch (NullPointerException e)
                 {
                 System.out.println("Inner catch"); }
}
catch (ArithmeticException e)
     {
System.out.println("Outer catch: Division by 0");
      }
   }
}
```

OUTPUT:

Outer try
Inner try
Outer catch: Division by zero



- The inner catch cannot handle ArithmeticException, so Java looks for a matching catch in the outer try.
- The outer catch matches it and handles the exception.
- If neither catch matched, the exception would eventually be handled by the Java runtime system

throw

- In Java, the throw keyword is **used to explicitly throw an exception** from the code.
- The general form of throw is shown here:

  throw ThrowableInstance;

  - Here, *ThrowableInstance* must be an object of type *Throwable* or a subclass of *Throwable*.
  - There are two ways to obtain a Throwable object:
    - using a parameter in a catch clause.

      or

    - creating one with the new operator.
- **The flow of execution stops immediately after the throw statement;** any subsequent statements are not executed.

- The nearest enclosing *try* block is inspected to see if it has a *catch* statement that matches the type of exception.
  - If it does find a match, control is transferred to that statement.
  - If not, then the next enclosing *try* statement is inspected, and so on.
  - If no matching *catch* is found, then the default exception handler halts the program and prints the stack trace.

Syntax:

throw new ExceptionType("Error message");

Eg:

```
public class Example {
    public static void checkAge(int age) {
        if (age < 18) {
      throw new IllegalArgumentException("Age must be 18 or above");
        }
        System.out.println("Eligible");
    }
    public static void main(String[] args) {
        checkAge(15);
    }
}
```

- IllegalArgumentException → exception class
- new IllegalArgumentException(...) → creates an exception object
- "Age must be 18 or above" → message stored in the exception object
- throw → throws that exception object

OUTPUT:

```
Exception in thread "main" java.lang.IllegalArgumentException:
Age must be 18 or above
```

throws

- A ***throws*** clause lists the types of exceptions that a method might throw.
- ie,
  - Suppose, a method is capable of causing an exception that it does not handle.
  - It must specify this behavior so that callers of the method can guard themselves against that exception.
  - This is done by including a *throws* clause in the method's declaration.
- *throws* tells the caller of the method that the exception should be handled by the caller.

Syntax:

```
returnType methodName(List of parameters) throws ExceptionType{

   // method body

}
```

Eg:

```
class Demo {
    void divide() throws ArithmeticException {
        int result = 10 / 0;
    }
    public static void main(String[] args) {
        Demo obj = new Demo();
        try {
            obj.divide();
        }
catch (ArithmeticException e) { System.out.println("Cannot divide by zero"); }}}
```

OUTPUT:

Cannot divide by zero

throw vs *throws* in Java:

Feature	`throw`	`throws`
Type	Keyword	Keyword
Used For	Throwing an exception explicitly	Declaring exceptions a method can throw
Location	Inside method body	In method signature
Example	`throw new IOException("File missing")`	`public void readFile() throws IOException`

finally

- The *finally* block in Java is used to **execute a block of code regardless of whether an exception occurs or not.**
- *finally* creates a block of code that will be **executed after a *try/catch* block has completed and before the code following the *try/catch* block.**
- *finally* can be useful for closing file handles and freeing up any other resources that might have been allocated at the beginning of a method with the intent of disposing of them before returning.
    - ie, this block ensures that necessary codes are not skipped because of exception.
- **The *finally* clause is optional.**
- However, **each *try* statement requires at least one *catch* or a *finally* clause.**

Syntax:

```
try {
    // code that may cause an exception
}
catch (Exception e) {
    // handling the exception
}
finally {
    // code that will always execute
}
```

Eg 1:

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
        finally {
            System.out.println("Finally block is executed");
        }
    }
}
```

OUTPUT:

```
Cannot divide by zero
Finally block is executed
```

Eg 2:

```
public class FinallyExample {
public static void main(String[] args) {
   try {
      int result = 10 / 2;
      System.out.println("Result = " + result);
   }
   catch (ArithmeticException e) {
    System.out.println("Exception occurred");
  }
finally {
        System.out.println("Finally block is executed");
      }
   }
}
```

OUTPUT:

Result = 5

Finally block is executed

Java's Built-in Exceptions

- Inside the standard package java.lang, Java defines several exception classes.
- The **checked exceptions defined in java.lang** are listed in the table below.

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

- This table lists the exceptions defined by java.lang that must be included in a method's *throws* list if that method can generate one of these exceptions and does not handle it itself. These are called *checked* exceptions.

- The **unchecked exceptions defined in java.lang** are listed in the table below;

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Custom Exceptions:

- Although Java's built-in exceptions handle most common errors, we may probably want to create our own exception types to handle situations specific to our applications.
- A custom exception is a **user-defined exception created by the programmer to handle application-specific error conditions.**
- Java allows us to create our own exception classes by extending either:
  - **Exception** → for a checked custom exception.
  - **RuntimeException** → for an unchecked custom exception.

PACKAGES & INTERFACES

Java's most innovative features: packages and interfaces.

PACKAGES:

- Packages are **containers for classes** that are used to **keep the class name space compartmentalized.**
- Packages help **organize Java programs** and provide **code reusability, access control,** and **easier maintenance.**
- Eg:
  - Suppose two programmers create a class with the same name, *List.*
  - Both classes are named *List,* but there is no conflict if they belong to different packages.
    - ie, A package allows to create a class named List, which you can store in your own package without concern that it will collide with some other class named List stored elsewhere.
  - Packages **prevent naming conflicts** by allowing classes with the same name to exist in different packages.

Using Identical Class Names with packages:

Eg 1:

```
package pkg1;

public class Test{

void display( )

{

System.ouy.println(“Package 1”);

}
```

Eg 2:

```
package pkg2;

public class Test{

void display( )

{

System.out.println(“Package 2”);

}
```

- Both classes are named as Test, but their fully qualified names are different:
  - pkg1.Test
  - pkg2.Test
- So, java treats them as two different classes.

Why Do We Need Packages in Java?

- In Java, classes are organized into packages to manage the class name space.
- Without packages, every class would need a unique name to avoid name collisions.
- As the number of classes increases, finding unique and meaningful names becomes difficult.
- Different programmers may choose the same class name, which can cause naming conflicts.
- **Packages divide the class name space into manageable groups,** allowing classes with the same name to exist in different packages.
- A package therefore acts as a **naming mechanism** that **helps avoid name collisions.**
- Packages also provide **visibility and access control.**
- Classes and class members can be made accessible only within the same package, when required.
- Thus, packages help organize classes, avoid naming conflicts, and control access to classes and their members.

Defining a Package

- To create a package, simply include a package command as the first statement in a Java source file.
- Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored.
- If you omit the package statement, the class names are put into the default package, which has no name.

General form of the package statement:

package pkg;

- Here, pkg is the name of the package

Eg:

```
package MyPackage;
```

- Java uses file system directories to store packages.
- Eg:
  - The .class files for any classes we declare to be part of MyPackage must be stored in a directory called MyPackage.
- It is also possible to create a hierarchy of packages.
  - To do so, simply separate each package name from the one above it by use of a period.
  - The general form of a multileveled package statement is;

```
package pkg1[.pkg2[.pkg3]];
```

- A package hierarchy must be reflected in the file system of our Java development system.
- Eg:
  - Suppose, a package is declared as

```
package java.awt.image;
```

  - This needs to be stored in java\awt\image in a Windows environment

CLASSPATH:

- **Java packages are mapped to directories (folders) in the file system.**
- The Java run-time system needs to know where to look for these package directories.
- Java uses **three ways to locate user-defined packages:**

1. Current Working Directory

  - By default, Java starts searching from the current working directory.
  - If the package directory is inside the current directory, Java can find it.

2. CLASSPATH Environment Variable

  - We can specify one or more directory paths using the CLASSPATH environment variable.
  - Java searches these paths to find the required package.

3. Using -classpath Option

  - The -classpath (-cp) option of java and javac can be used to specify the location of classes and packages.

A Short Package Example:

```
// A simple package
package MyPack;

class Balance {
  String name;
  double bal;

  Balance(String n, double b) {
    name = n;
    bal = b;
  }

  void show() {
    if(bal<0)
      System.out.print("--> ");
    System.out.println(name + ": $" + bal);
  }
}

class AccountBalance {
  public static void main(String args[]) {
    Balance current[] = new Balance[3];

    current[0] = new Balance("K. J. Fielding", 123.23);
    current[1] = new Balance("Will Tell", 157.02);
    current[2] = new Balance("Tom Jackson", -12.33);

    for(int i=0; i<3; i++) current[i].show();
  }
}
```

OUTPUT:

K. J. Fielding: $123.23

Will Tell: $157.02

--> Tom Jackson: $-12.33

- Call this file *AccountBalance.java* and put it in a directory called *MyPack.*
- Next, compile the file.
- Make sure that the resulting .class file is also in the *MyPack* directory.
- Then, try executing the *AccountBalance* class, using the following command line:

    java MyPack.AccountBalance

- When we execute this command, will need to be in the directory above MyPack.
- *AccountBalance* is now part of the package *MyPack*.
- This means that it cannot be executed by itself.
    - ie, we cannot use the command line:

        java AccountBalance

- *AccountBalance* must be qualified with its package name.

Access Protection

- Access control determines who can access classes and class members such as variables and methods.
- It helps achieve data hiding and security.
- Java provides different levels of visibility using the following keywords;
    - **private**, **public**, **protected**, **default (no access specifier)**
- **A package is a container for classes and sub-packages.**
- **A class is a container for:**
    - **Data (variables)**
    - **Code (methods)**

- **Packages** provide an **additional level of access control.**
- They **help organize classes and control their visibility.**

Key idea:

- Class → contains data and code
- Package → contains classes
- Java considers access based on the relationship between classes:
    - **Subclass in the same package**
    - **Non-subclass in the same package**
    - **Subclass in a different package**
    - **Non-subclass in a different package**
- These categories determine whether a class member can be accessed.

- Following table shows for levels of member access in Java;

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Note: *protected* access in a different package is through inheritance/subclass relationship.

Private Members:

- Accessible only within the same class
- Cannot be accessed directly by:
  - Other classes in the same package
  - Subclasses
  - Classes in other packages

ie, **private → Same class only**

Default Access:

- No access specifier is written.
- Accessible within the same package.
- Accessible by:
  - Classes in the same package
  - Subclasses in the same package
- Not accessible from a different package.

ie, **default → Same package**

Protected Members:

- Accessible within the same package
- Also accessible in subclasses from different packages
- Provides more access than default.
- Useful when inheritance is involved.

ie, **protected → Same package + subclasses in other packages**

Public Access:

- Can be accessed from anywhere
- Accessible by:
  - Same class
  - Same package
  - Subclasses
  - Classes in other packages

ie, **public → Everywhere**

In increasing order of access: private → default → protected → public

Access Control Example

```
class Student {

    private int marks;

    int rollNo;             // default

    protected String name;

    public String college;

}
```

- marks → accessible only inside Student
- rollNo → accessible within the same package
- name → same package + subclasses
- college → accessible from anywhere

IMPORTING PACKAGES:

- The ***import*** statement in Java is used to make classes or interfaces from other packages available for direct use in a program.
- Once a class is imported, you can refer to it using only its class name, instead of writing its complete package name.

General Syntax:

```
import packageName.className;

        Or

import packageName.*;
```

- packageName → Name of the package.
- className → Specific class to be imported.
- * → Imports all accessible classes/interfaces in that package.

1. Importing a specific class:

import java.util.Date;

Now, we can write;

```
Date d = new Date();
```

instead of java.util.Date d = new java.util.Date();

2. Importing an entire package:

import java.io.*;

This makes the accessible classes/interfaces in the java.io package available for direct use.

Position of import Statement:

- In a Java source file, the order is generally:

```
package MyPack;      // Optional
import java.util.Date;
import java.io.*;
class Demo {
   // class definition
}
```

- So, the import statement must appear after the package statement (if present) and before any class or interface definitions.

Note:

- The star form may increase compilation time—especially if we import several large packages.
- For this reason it is a good idea to explicitly name the classes that you want to use rather than importing whole packages.
- All of the standard Java classes included with Java are stored in a package called java.
- The basic language functions are stored in a package inside of the java package called java.lang.
- Normally, you have to import every package or class that you want to use, but since Java is useless without much of the functionality in java.lang, it is implicitly imported by the compiler for all programs.
- This is equivalent to the following line being at the top of all of your programs:

```
import java.lang.*;
```

INTERFACES:

- In Java, an interface is a **blueprint that describes what a class should do, without requiring the interface itself to provide all the implementation details.**
- ie, using interface, you **can specify what a class must do, but not how it does it.**
- Interfaces are **syntactically similar to classes**, but they **lack instance variables**, and their **methods are declared without any body.**
- **By providing the interface keyword, Java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.**
- **An interface defines a contract that must be followed by the classes that implement it.**

Eg 1:

```
interface Animal {
    void makeSound();
}
class Dog implements Animal {
    public void makeSound() {
        System.out.println(“Bow");
    } }
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
    } }
```

OUTPUT:

Bow

- The important idea is that;
  Animal defines a contract - anything implementing *Animal* must provide the required makeSound() method.
- The main advantage of using an interface in the *Animal* program is that, it allows different classes to follow the same contract while providing their own implementation.

Why interfaces are useful:

- Interfaces **provide abstraction** and allow unrelated classes to follow the same contract.
- Eventhough interfaces provide a common contract, it **allows different classes to provide their own implementations.**
- They also **let Java support a form of multiple inheritance**, because one class can implement several interfaces.
- An interface allows a single interface reference to refer to objects of different implementing classes, **enabling runtime polymorphism and flexible method behavior.**

Eg 2:

```
interface Flyable {
    void fly();
}
interface Swimmable {
    void swim();
}
class Duck implements Flyable, Swimmable {
    public void fly() {
        System.out.println("Duck is flying");
    }
    public void swim() {
        System.out.println("Duck is swimming");
    }
}
public class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}
```

OUTPUT:

Duck is flying

Duck is swimming

- Flyable defines the fly() method.
- Swimmable defines the swim() method.
- Duck implements both interfaces using:

```
class Duck implements Flyable, Swimmable
```

- Therefore, Duck must provide implementations for both fly() and swim().
- This demonstrates that, **Java allows a class to implement multiple interfaces, providing a way to achieve multiple inheritance of behavior/contracts.**
- **ie, Java supports multiple inheritance through interfaces.**

Defining an Interface:

- An interface is defined much like a class.
- This is the general form of an interface:

```
access interface name {
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    type final-varname1 = value;
    type final-varname2 = value;
    // ...
    return-type method-nameN(parameter-list);
    type final-varnameN = value;
}
```

- When no access specifier is included, then default access results => the interface is only available to other members of the same package.
- When the access specifier is public, the interface can be used by any other code.
- *name* is the name of the interface.
- The methods that are declared have no bodies.
- They end with a semicolon after the parameter list.
- Variables can be declared inside of interface declarations.
    - They are implicitly final and static, ie, they can't be changed by the implementing class.
    - They must also be initialized.
    - All methods and variables are implicitly public.

IMPLEMENTING INTERFACES:

- **Once an interface has been defined, one or more classes can implement that interface.**
- To implement an interface, include the ***implements*** clause in a class definition, and then create the methods defined by the interface.
- The general form of a class that includes the implements clause looks like this:

```
class classname [extends superclass] [implements interface [,interface...]]
{
// class-body
}
```

- If a class implements more than one interface, the interfaces are separated with a comma.
- If a class implements two interfaces that declare the same method, then the same method will be used by clients of either interface.
- When you implement an interface method, it must be declared as **public.**
- Also, the type signature of the implementing method must match exactly the type signature specified in the interface definition.
- **Eg 1:**

```
interface Callback {
  void callback(int param);
}

class Client implements Callback {
  // Implement Callback's interface
  public void callback(int p) {

    System.out.println("callback called with " + p);
  }
}
```

- It is permissible for classes that implement interfaces to define additional members of their own.
- Eg:

Here, nonIfaceMeth( ) is an additional method which is defined within the class Client.

```
interface Callback {
  void callback(int param);
}

class Client implements Callback {
  // Implement Callback's interface
  public void callback(int p) {
    System.out.println("callback called with " + p);
  }

  void nonIfaceMeth() {
    System.out.println("Classes that implement interfaces " +
                       "may also define other members, too.");
  }
}
```

Accessing Implementations Through Interface References

- You can declare variables as object references that use an interface rather than a class type.
- Any instance of any class that implements the declared interface can be referred to by such a variable.
- When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to.
- This is one of the key features of interfaces.
- The method to be executed is looked up dynamically at run time, allowing classes to be created later than the code which calls methods on them.
- The calling code can dispatch through an interface without having to know anything about the “callee.”

Eg 1:

```
interface Callback {
  void callback(int param);
}

class Client implements Callback {
  // Implement Callback's interface
  public void callback(int p) {
    System.out.println("callback called with " + p);
  }

  void nonIfaceMeth() {
    System.out.println("Classes that implement interfaces " +
                       "may also define other members, too.");
  }
}

class TestIface {
  public static void main(String args[]) {
    Callback c = new Client();
    c.callback(42);
  }
}
```

OUTPUT:

callback called with 42

- The above example calls the callback( ) method via an interface reference variable.
- Note that variable *c* is declared to be of the interface type *Callback*, yet it was assigned an instance of Client.
    - This demonstrates the advantage of an interface: polymorphism.
    - ie, the reference variable ‘c’ of type Callback (which is an Interface) pointed to Client (which is a class) object.
- Although c can be used to access the callback( ) method, it cannot access any other members of the Client class.
- **An interface reference variable only has knowledge of the methods declared by its interface declaration**.
- Thus, c could not be used to access nonIfaceMeth( ) since it is defined by Client but not Callback.

Comparison of Java Interface/Class References

Statement	Valid?	Reference Type	Object Type	Reason
Client c = new Client();	YES	Client	Client	A class can create its own object.
Callback c = new Client();	YES	Callback	Client	Client implements Callback; supports runtime polymorphism.
Client c = new Callback();	NO	Client	—	An interface cannot be instantiated directly; a Callback object cannot be assigned to Client.
Callback c = new Callback();	NO	Callback	—	An interface cannot be instantiated directly using new.

Key point: Callback c = new Client(); is used to demonstrate interface-based runtime polymorphism.

Eg2:

```
interface Callback {
  void callback(int param);
}

class Client implements Callback {
  // Implement Callback's interface
  public void callback(int p) {
    System.out.println("callback called with " + p);
  }

  void nonIfaceMeth() {
    System.out.println("Classes that implement interfaces " +
                       "may also define other members, too.");
  }
}

// Another implementation of Callback.
class AnotherClient implements Callback {
  // Implement Callback's interface
  public void callback(int p) {
    System.out.println("Another version of callback");
    System.out.println("p squared is " + (p*p));
  }
}
```

```
class TestIface2 {
  public static void main(String args[]) {
    Callback c = new Client();
    AnotherClient ob = new AnotherClient();

    c.callback(42);

    c = ob; // c now refers to AnotherClient object
    c.callback(42);
  }
}
```

OUTPUT:

callback called with 42

Another version of callback

p squared is 1764

Extending Interfaces:

- One interface can inherit another by use of the keyword *extends*.
- The syntax is the same as for inheriting classes.
- **When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.**

Eg:

```
// One interface can extend another.
interface A {
  void meth1();
  void meth2();
}

// B now includes meth1() and meth2() -- it adds meth3().
interface B extends A {
  void meth3();
}

// This class must implement all of A and B
class MyClass implements B {
  public void meth1() {
    System.out.println("Implement meth1().");
  }

  public void meth2() {
    System.out.println("Implement meth2().");
  }

  public void meth3() {
```

```
    System.out.println("Implement meth3().");
  }


}

class IFExtend {
  public static void main(String arg[]) {
    MyClass ob = new MyClass();

    ob.meth1();
    ob.meth2();
    ob.meth3();
  }
}
```

OUTPUT:

Implement meth1().
Implement meth2().
Implement meth3().